

Transcrição do Material interativo: Escalonamento de Processos

Página 1

A multiprogramação é o recurso que permite que diversos processos estejam em memória para execução concorrente ou paralela em sistemas operacionais. É o conceito-chave em sistemas operacionais.

Página 2

A multiprogramação é o que nos permite utilizar o processador em sua capacidade máxima. Isso é especialmente importante não apenas para aumentar a vazão e executar mais programas em um intervalo de tempo, mas porque, em geral, os programas intercalam durante o uso do processador e dos dispositivos de entrada e saída. Neste caso, é inteligente e esperado que, enquanto o processo está preso em uma operação de I/O, o processador seja utilizado por outro processo na fila do sistema operacional.

Página 3

Temos aqui um exemplo em que um processo intercala entre essas duas fases: a fase em que o processador está sendo estressado *CPU burst*; e a fase em que os dispositivos de I/O estão sendo estressados *I/O burst*.

Página 4

Para aproveitar de modo adequado os recursos do computador e para gerenciar essas fases, algoritmos de escalonamento devem ser implementados de modo a selecionar da fila de aptos processos para serem executados. O escalonador, então, entra em ação quando há a mudança entre os estados básicos do processo.

Página 5

E quais são os critérios para definir a ordem do escalonamento? É possível que a gente queira maximizar a utilização do processador, deixando-o ocupado por mais tempo; maximizar a quantidade de processos que finalizam sua execução por unidade de tempo; minimizar a quantidade de tempo necessário para a execução dos processos; minimizar a quantidade de tempo que os processos esperam na fila de processos aptos; e também minimizar o tempo de resposta inicial em um sistema de tempo compartilhado, por exemplo.

Página 6

O primeiro algoritmo, então, é o *First Come, First Served*, e é o primeiro a chegar. O primeiro a ser atendido é o FIFO, que é um algoritmo que respeita a ordem de chegada na fila e não usa outro critério para decisão do escalonamento. Nesse diagrama, nós vemos os tempos de processamento de cada processo: o processo 1 leva 24 unidades de tempo para executar; o processo 2, 3 unidades de tempo; e o processo 3, 3 unidades de tempo. O tempo

de espera de cada processo na fila é a soma dos tempos de execução dos processos anteriores, como podemos ver no exemplo.

Página 7

Agora, vejam que o tempo de espera médio é diferente para tempos de processamentos diferentes. Pensem agora sobre o desempenho do *First Come, First Served* em uma situação dinâmica. Imaginem que há um processo limitado por processador e muitos processos limitados por I/O. Quantos processos fluem pelo sistema? O resultado pode ser o seguinte: o processo limitado por CPU ocupará e manterá o processador. Durante esse período, todos os outros processos terminarão suas tarefas de I/O e entrarão na fila de aptos, esperando pelo processador.

Enquanto os processos esperam na fila de aptos, os dispositivos de I/O ficam ociosos, eventualmente o processo limitado por processador termina seu pico de CPU e passa para um dispositivo de I/O. Todos os processos limitados por I/O tem picos de CPU curtos, são executados rapidamente e voltam para as filas de I/O. Nesse momento, a CPU permanece ociosa. O processo limitado por CPU volta, então, para a fila de prontos ou de aptos, e o processador é alocado para ele. Novamente, todos os processos limitados por I/O terão que esperar na fila de prontos, até que o processo limitado por CPU termine. Nesse caso há um efeito comboio, pois todos os outros processos esperam que o grande processo desocupe o processador. Esse efeito resulta em uma baixa utilização do processador e dos dispositivos, diferente de uma situação em que processos mais curtos são atendidos primeiro.

Página 8

O segundo algoritmo de escalonamento é o *Shortest Job First*, menor Job primeiro. É o algoritmo de prioridade, que opta pelos escalonamentos de processos que executam em menor tempo. O grande problema aqui é determinar, antes de executar, o tempo de execução desse processo.

Página 9

Aqui há o tempo médio de espera do algoritmo. Neste caso, podemos ver que o uso do processador ou o tempo de execução de cada processo é dado na tabela: o processo 1 tem duração de 6 unidades de tempo; o processo 2, tem 8 unidades de tempo; o processo 3, tem 7 unidades; e o processo 4, tem 3 unidades de tempo. Temos, então, o diagrama em que os processos são executados. Vejam que aqueles de duração menor são executados primeiro, ou seja, primeiro é executado o processo 4; na sequência, o processo 1; depois, o processo 3; e, por último, o processo 2. Diante disso, o tempo de espera médio é 7.

Página 10

O terceiro algoritmo de escalonamento é o *Round Robin*. Ele implementa uma fila circular em que o processo que pede o uso da CPU é enviado para o final da fila. Nesse algoritmo, existe o conceito de *quantum*, um análogo ao *timer*, em que define-se um tempo de uso do processador pelos processos antes que tenham sua execução interrompida ou *preemptada*. Um *quantum* relativamente alto faz com que o algoritmo tenha um

comportamento parecido com o *First Come, First Served*. Um *quantum* pequeno faz com que a sobrecarga decorrente das trocas de contexto dos processos seja alta.

Página 11

Vemos aqui um exemplo de *quantum* com tamanho 4. Nesta situação, o processo 1 tem tempo de duração 24; o processo 2, tem 3; e o processo 3, tem 3. Vejam que no diagrama eles irão executar respeitando o tamanho do *quantum*, que é 4. Então, o processo 1 é executado por 4 unidades de tempo e, na sequência, o processo 2. Ele não vai usar 4 unidades de tempo, porque o tempo total dele é 3. Da mesma forma ocorre com o processo 3. Como apenas o processo 1 ainda precisa ser executado, apenas ele executará em *quantum* de tamanho 4 até que a execução seja finalizada.

Página 12

Neste caso, o funcionamento é o mesmo, mas a diferença é que o *quantum* tem tamanho 20.

Página 13

Temos uma imagem que relaciona o tempo do *quantum* e a quantidade de trocas de contexto. Aqui, nós vemos um tempo de processo igual a 10. Na primeira faixa, no primeiro retângulo, vemos um *quantum* igual a 12. Se o tempo do processo é 10, então, não haverão trocas de contexto, ou seja, trocas de contexto = 0. No segundo exemplo, onde temos o retângulo azul, vemos que o tamanho do *quantum* é igual a 6. Neste caso teremos uma única troca de contexto. No terceiro exemplo, vemos que o *quantum* tem tamanho 1, então, serão necessárias 9 trocas de contexto para finalizar a execução do processo.

Página 14

Agora, podemos pensar em cenários em que há múltiplas filas e, para cada fila, um algoritmo de escalonamento diferente aplicado. Isso é particularmente importante quando se trabalha com algoritmos de prioridades, porque nesta situação, processos com baixa prioridade podem sempre perder o processador para processos com prioridades mais altas, não conseguindo finalizar suas execuções.